

Shared CLI option parser + **AUTOLISP-...** installer

User Manual

clautolisp contributors

May 25, 2026

Contents

1	Introduction	1
2	Loading	1
3	Anatomy of an option	1
4	The common option set	2
5	Building a tool's parser	2
6	Installing the AUTOLISP-... globals	2
7	Encoding resolution	3
8	The cli-usage-error condition	3
9	See also	3
10	Synchronisation	4

1 Introduction

This manual is the programmer's guide to `clautolisp.autolisp-cli`. The audience is a clautolisp developer adding a CLI option to either `clautolisp(1)` or `alfe(1)`, or writing a new tool that wants the same standard surface.

End-users of the existing CLIs don't need this manual — the `clautolisp(1)` and `alfe(1)` man pages and `--help` banners cover the user-facing surface.

2 Loading

(asdf:load-system "clautolisp/autolisp-cli")

Dependency: `clautolisp/autolisp-runtime` (used by the transmit installer to call `set-variable`).

3 Anatomy of an option

Each CLI option is one `option-spec` record:

```
(make-option-spec :longs '("-load") :shorts '("-l") :takes-arg-p t :handler
(lambda (opts value name) (declare (ignore name)) (setf (cli-options-actions
opts) (append (cli-options-actions opts) (list (cons :file value))))))
```

The HANDLER receives:

- `OPTS` — the cli-options struct being populated.
- `VALUE` — the consumed argument when `TAKES-ARG-P`, nil otherwise.
- `NAME` — the long or short form actually seen, useful in error messages.

The handler mutates `OPTS` directly (typically via `SETF` on a slot).

4 The common option set

`*common-option-specs*` covers everything both tools accept: `--help` / `-h`, `--version` / `-V`, `--list-encodings`, `--quiet` / `-q`, `--verbose` / `-v`, `--debug` / `-d`, `--dialect`, the dialect shorthands (`--strict` / `--autocad` / `--bricscad` / `--clautolisp`), `--host`, `--load` / `-l`, `--eval` / `-x`, `--interactive` / `-i`, `-e`, `-E`, `--no-init` / `-norc`, `--no-color`.

Each tool prepends or appends its own tool-specific specs. `alfe` adds `--mode`, `--backend`, `--dwg`, `--epure`, `--bootstrap-phase`, `--workdir`, `--keep-workdir`, `--timeout`, `--dry-run`, `--main`, `--quit`, plus conflict-checking wrappers for the dialect shorthands. `clautolisp` adds `--mock-input`, `--gui`, `--trace`.

5 Building a tool's parser

```
(defun my-tool/parse-arguments (argv) (let* ((specs (append my-tool-option-specs common-option-specs)) (options (make-cli-options))) (parse-arguments-with-spec specs argv :initial-options options) options))
```

The first-match-wins lookup means specs you prepend take priority over the common ones; this is how `alfe` wraps the dialect shorthands with mutual-exclusion checking.

6 Installing the AUTOLISP-... globals

After the runtime context is built (and before init files load), install the transmit-options globals:

```
(install-transmit-variables context (cli-options->transmit-bindings options :backend "MYTOOL" :version-text version :usage-text (usage-string)))
```

For `alfe`, use `alfe.cli:cli-options-transmit-bindings-for-alfe` instead — it handles the action-object cons translation.

For dynamic-scope variables (`*AUTOLISP-LOAD-PATHNAME*`, `*AUTOLISP-EXPRESSION*`, `*AUTOLISP-INTERACTIVE*`), wrap each action with:

```
(call-with-dynamic-transmit-binding context "AUTOLISP-LOAD-PATHNAME" (make-autolisp-string (namestring path)) (lambda () (autolisp-load-file-in-context path context ...)))
```

7 Encoding resolution

To validate a user-typed encoding string, call:

```
(canonical-encoding-name "utf-8" "-e") ;; => "UTF-8" (encoding-keyword "Utf-8-mac" "-e") ;; => (:UTF-8 :NEWLINE :CR)
```

Both signal `cli-usage-error` on typos. The validator probes the running CL implementation's external-format registry on SBCL / CCL, so even alphabetically-plausible-but-wrong names are caught.

To enumerate every encoding the impl handles, call `print-encodings` (the implementation behind `--list-encodings`).

8 The cli-usage-error condition

```
(define-condition cli-usage-error (simple-error) ((option :initarg :option :reader cli-usage-error-option) (message :initarg :message :reader cli-usage-error-message)))
```

Raise it directly from a value parser to surface a clean message to the user:

```
(error 'cli-usage-error :option "--mode" :message "Unknown mode 'foo'
(expected auto/automation/batch)" )
```

9 See also

- autolisp-cli-specifications.org — design + internal invariants.
- [issues/closed/transmit-options.issue](#) — the **AUTOLISP-...** contract.
- [issues/closed/encodings.issue](#) — the encoding registry rules.
- [issues/closed/line-termination.issue](#) — line-terminator suffix.
- [issues/closed/source-aware-defun-documentation.issue](#) — future work that will add a documentation tier to the transmit installer.

10 Synchronisation

Per AGENTS.md, code / spec / user-manual changes propagate.